

BASES DE DATOS II

Guía Completa de Teoría

Desde el modelo relacional hasta Triggers y Transacciones

Parcial: 12 de Junio de 2025

PARTE 1: FUNDAMENTOS DE BASES DE DATOS

1.1 ¿Qué es una Base de Datos Relacional?

Una base de datos relacional organiza los datos en tablas (también llamadas relaciones). Cada tabla tiene filas (registros) y columnas (atributos). Las tablas se relacionan entre sí a través de claves.

Concepto	Descripción	Ejemplo
Tabla / Relación	Estructura que almacena datos del mismo tipo.	clientes, productos, pedidos
Fila / Registro / Tupla	Un dato individual dentro de la tabla.	El cliente 'Ana García'
Columna / Atributo	Una característica de la entidad.	nombre, email, precio
Clave Primaria (PK)	Identifica de forma única a cada fila.	id_cliente INT AUTO_INCREMENT
Clave Foránea (FK)	Referencia a la PK de otra tabla.	id_categoria en productos
Constraint / Restricción	Regla que garantiza integridad de datos.	NOT NULL, UNIQUE, FOREIGN KEY

1.2 Tipos de Relaciones entre Tablas

Tipo	Descripción	Ejemplo en tienda_online
1 a 1 (1:1)	Cada fila de A se relaciona con exactamente una fila de B.	Un cliente — un perfil de usuario
1 a muchos (1:N)	Una fila de A se relaciona con muchas filas de B.	Una categoría — muchos productos
Muchos a muchos (N:M)	Requiere tabla intermedia (tabla de unión).	Pedidos ↔ Productos via detalle_pedido

1.3 Integridad Referencial

La integridad referencial garantiza que los valores de una clave foránea siempre correspondan a un valor existente en la tabla referenciada. MySQL la aplica automáticamente cuando se define FOREIGN KEY en tablas InnoDB.

```
FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)
-- Si intentás insertar un producto con id_categoria = 99
(inexistente), MySQL lo rechaza.
```

```
-- Si intentás borrar una categoría que tiene productos, MySQL lo  
rechaza (por defecto).
```

PARTE 2: DDL — Definición de Estructura

DDL (Data Definition Language): instrucciones que crean o modifican la estructura de la base de datos.

2.1 CREATE TABLE

Sintaxis básica con los tipos de datos y restricciones más comunes:

```
CREATE TABLE IF NOT EXISTS productos (  
    id_producto SMALLINT(5) PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(100) NOT NULL,  
    precio DECIMAL(10,2) NOT NULL,  
    stock TINYINT(2) DEFAULT 0,  
    id_categoria TINYINT(2),  
    activo TINYINT(1) DEFAULT 1,  
    FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria)  
);
```

Tipo de dato	Uso	Ejemplo
INT / SMALLINT / TINYINT	Números enteros. TINYINT(1) se usa como booleano.	id_producto SMALLINT(5)
DECIMAL(p,s)	Números con decimales exactos. p = dígitos totales, s = decimales.	precio DECIMAL(10,2)
VARCHAR(n)	Texto de longitud variable hasta n caracteres.	nombre VARCHAR(100)
DATE	Fecha sin hora: YYYY-MM-DD.	fecha_registro DATE
DATETIME	Fecha y hora: YYYY-MM-DD HH:MM:SS.	fecha_modificacion DATETIME
ENUM(...)	Lista fija de valores permitidos.	estado ENUM('pendiente','enviado','entregado')

2.2 ALTER TABLE — Modificar estructura

Se usa para agregar, modificar o eliminar columnas en una tabla existente:

```
-- Agregar columna:  
ALTER TABLE clientes ADD COLUMN telefono VARCHAR(20) NULL;  
ALTER TABLE clientes ADD COLUMN ciudad VARCHAR(60) NULL;  
  
-- Modificar tipo o restricción de una columna:  
ALTER TABLE productos MODIFY COLUMN stock SMALLINT(4) DEFAULT 0;
```

```
-- Eliminar columna:  
ALTER TABLE clientes DROP COLUMN telefono;  
  
-- Agregar índice:  
ALTER TABLE clientes ADD INDEX idx_email (email);
```

2.3 DROP y TRUNCATE

```
DROP TABLE IF EXISTS tabla_vieja;    -- Elimina la tabla y todos sus  
datos (irreversible)  
TRUNCATE TABLE logs;                  -- Elimina todos los registros,  
conserva la estructura
```

⚠ Importante: DROP elimina la tabla por completo. TRUNCATE vacía los datos pero mantiene la tabla. Ninguno puede deshacerse con ROLLBACK en MySQL.

PARTE 3: DML — Manipulación de Datos

DML (Data Manipulation Language): instrucciones para insertar, consultar, actualizar y eliminar registros.

3.1 INSERT

```
-- Insertar un registro:
INSERT INTO clientes (nombre, email, fecha_registro)
VALUES ('Ana García', 'ana@email.com', '2024-01-15');

-- Insertar múltiples registros a la vez:
INSERT INTO productos (nombre, precio, stock, id_categoria) VALUES
('Auriculares Bluetooth', 3500.00, 20, 1),
('Smartwatch', 8900.00, 5, 1),
('Remera básica', 950.00, 50, 2);
```

3.2 SELECT — Consulta

```
-- Básico:
SELECT nombre, precio FROM productos WHERE stock > 0;

-- Con alias y cálculo:
SELECT nombre, precio * 1.21 AS precio_con_iva FROM productos;

-- Con ORDER BY y LIMIT:
SELECT * FROM productos ORDER BY precio DESC LIMIT 5;

-- Con funciones de agregación:
SELECT COUNT(*) AS total_productos, AVG(precio) AS precio_promedio FROM
productos;

-- Con GROUP BY:
SELECT id_categoria, COUNT(*) AS cantidad, SUM(stock) AS stock_total
FROM productos
GROUP BY id_categoria;

-- HAVING (filtro sobre grupos, NO sobre filas individuales):
SELECT id_categoria, SUM(stock) AS stock_total
FROM productos
GROUP BY id_categoria
HAVING stock_total > 20;
```

❏ **Tip:** WHERE filtra filas individuales ANTES de agrupar. HAVING filtra grupos DESPUÉS de GROUP BY. No son intercambiables.

3.3 UPDATE y DELETE

```
-- Actualizar registros:
UPDATE clientes SET ciudad = 'Rosario' WHERE id_cliente = 1;
UPDATE productos SET precio = precio * 1.10 WHERE id_categoria = 1;  --
10% de aumento

-- Eliminar registros:
DELETE FROM clientes WHERE id_cliente = 4;
DELETE FROM pedidos WHERE estado = 'pendiente' AND fecha_pedido <
'2024-01-01';
```

⚠ Importante: Siempre incluí WHERE en UPDATE y DELETE. Sin WHERE, la instrucción afecta TODAS las filas de la tabla.

PARTE 4: JOINS — Consultas entre Tablas

Los JOINS combinan filas de dos o más tablas basándose en una condición de unión (generalmente FK = PK).

4.1 Tipos de JOIN

Tipo	Devuelve	Cuándo usarlo
INNER JOIN	Solo las filas que tienen coincidencia en AMBAS tablas.	Cuando solo querés datos relacionados en las dos tablas.
LEFT JOIN	Todas las filas de la tabla izquierda + coincidencias de la derecha (NULL si no hay).	Cuando querés TODOS los registros de la tabla principal, aunque no tengan relación.
RIGHT JOIN	Todas las filas de la tabla derecha + coincidencias de la izquierda.	Menos común. Equivale a invertir el LEFT JOIN.

4.2 Ejemplos Prácticos

INNER JOIN — pedidos con datos del cliente:

```
SELECT p.id_pedido, c.nombre, p.fecha_pedido, p.estado,
       SUM(dp.cantidad * dp.precio_unitario) AS total
FROM   pedidos      p
INNER JOIN clientes  c ON c.id_cliente = p.id_cliente
INNER JOIN detalle_pedido dp ON dp.id_pedido = p.id_pedido
GROUP BY p.id_pedido, c.nombre, p.fecha_pedido, p.estado
ORDER BY total DESC;
```

LEFT JOIN — clientes con o sin pedidos (incluye los que nunca compraron):

```
SELECT c.nombre, COUNT(p.id_pedido) AS cantidad_pedidos
FROM   clientes c
LEFT JOIN pedidos p ON p.id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nombre;
```

LEFT JOIN para encontrar registros SIN relación (patrón IS NULL):

```
-- Productos que nunca fueron pedidos:
SELECT pr.nombre, pr.precio
FROM   productos      pr
LEFT JOIN detalle_pedido dp ON dp.id_producto = pr.id_producto
WHERE dp.id_producto IS NULL;
```

□ **Tip:** El patrón LEFT JOIN ... WHERE tabla_derecha.col IS NULL es la forma más eficiente de encontrar registros sin relación. Alternativa: NOT IN (subconsulta).

PARTE 5: VISTAS (VIEWS)

Una vista es una consulta guardada que se comporta como una tabla virtual. No almacena datos, los obtiene en tiempo real al ser consultada.

5.1 Crear y Reemplazar una Vista

```
CREATE OR REPLACE VIEW RESUMEN_CLIENTES AS
SELECT
    c.nombre AS
nombre_cliente,
    c.ciudad,
    COUNT(DISTINCT p.id_pedido) AS
cantidad_pedidos,
    COALESCE(SUM(dp.cantidad * dp.precio_unitario), 0) AS monto_total
FROM
    clientes c
LEFT JOIN
    pedidos p ON p.id_cliente = c.id_cliente
LEFT JOIN
    detalle_pedido dp ON dp.id_pedido = p.id_pedido
GROUP BY
    c.id_cliente, c.nombre, c.ciudad
ORDER BY
    monto_total DESC;
```

5.2 Usar una Vista

Una vez creada, la vista se consulta igual que una tabla:

```
SELECT * FROM RESUMEN_CLIENTES;
SELECT * FROM RESUMEN_CLIENTES WHERE monto_total > 20000;
SELECT * FROM RESUMEN_CLIENTES WHERE cantidad_pedidos = 0;
SELECT * FROM RESUMEN_CLIENTES WHERE ciudad = 'Rosario';
SELECT * FROM RESUMEN_CLIENTES WHERE cantidad_pedidos > 1 AND
monto_total > 20000;
```

5.3 Ventajas de las Vistas

- Simplifican consultas complejas: escribís la lógica una sola vez.
- Seguridad: podés dar acceso a la vista sin exponer las tablas base.
- Mantenibilidad: si cambia la lógica, solo modificás la vista.

□ **Recordá:** COALESCE(expr, 0) devuelve 0 si expr es NULL. Útil cuando un cliente no tiene pedidos y SUM() devuelve NULL.

PARTE 6: SUBCONSULTAS

Una subconsulta es una consulta anidada dentro de otra. Se puede usar en SELECT, FROM, WHERE o HAVING.

6.1 Subconsulta en WHERE

```
-- Productos que nunca fueron pedidos (forma alternativa al LEFT JOIN):
SELECT nombre, precio FROM productos
WHERE id_producto NOT IN (
    SELECT DISTINCT id_producto
    FROM    detalle_pedido
    WHERE   id_producto IS NOT NULL
);

-- Clientes que hicieron al menos un pedido:
SELECT nombre FROM clientes
WHERE id_cliente IN (SELECT DISTINCT id_cliente FROM pedidos);
```

6.2 Subconsulta Escalar (devuelve un único valor)

```
-- Productos con precio mayor al promedio:
SELECT nombre, precio FROM productos
WHERE precio > (SELECT AVG(precio) FROM productos);
```

□ **Tip:** Cuando una subconsulta puede devolver NULL (lista vacía en NOT IN), el resultado puede ser incorrecto. Siempre filtra con WHERE col IS NOT NULL dentro de la subconsulta.

PARTE 7: TRIGGERS EN MySQL

Un Trigger (disparador) es un objeto de base de datos asociado a una tabla que se ejecuta automáticamente cuando ocurre un evento DML (INSERT, UPDATE o DELETE). No se invocan manualmente; el motor los activa de forma implícita.

7.1 Casos de Uso

- Auditar cambios en tablas críticas (historial de modificaciones).
- Mantener columnas calculadas o resúmenes actualizados automáticamente.
- Implementar restricciones complejas que no puede expresar una constraint simple.
- Sincronizar datos entre tablas relacionadas.
- Validar reglas de negocio antes de confirmar una operación.

7.2 Los 6 Tipos de Triggers

Tipo	Momento	Cuándo se ejecuta	Caso de uso típico
BEFORE INSERT	Antes	Antes de insertar la fila	Validar o modificar valores antes de guardar
AFTER INSERT	Después	Después de insertar la fila	Registrar en auditoría, actualizar totales, descontar stock
BEFORE UPDATE	Antes	Antes de actualizar la fila	Verificar reglas de negocio, formatear, calcular valores
AFTER UPDATE	Después	Después de actualizar la fila	Log de cambios, sincronizar tablas
BEFORE DELETE	Antes	Antes de eliminar la fila	Guardar copia en log, evitar borrados no autorizados
AFTER DELETE	Después	Después de eliminar la fila	Restaurar stock, limpiar dependencias

7.3 Sintaxis Completa

```
DELIMITER $$
CREATE TRIGGER nombre_trigger
    {BEFORE | AFTER} {INSERT | UPDATE | DELETE}
    ON nombre_tabla
    FOR EACH ROW
BEGIN
    -- Cuerpo del trigger
    -- Podés usar: IF, CASE, variables locales, SIGNAL
    -- NO podés usar: COMMIT, ROLLBACK, START TRANSACTION
END$$
```

DELIMITER ;

7.4 Referencias NEW y OLD

Referencia	INSERT	UPDATE	DELETE
NEW.columna	Valor que se va a insertar	Valor nuevo (modificable en BEFORE)	No disponible (NULL)
OLD.columna	No disponible (NULL)	Valor original antes del cambio	Valor que se está eliminando

❏ **Clave:** En un BEFORE trigger podés asignar NEW.columna = valor para cambiar lo que se guarda en la tabla. En AFTER ya no podés porque la fila ya fue escrita.

7.5 SIGNAL SQLSTATE — Abortar una Operación

Para rechazar una operación desde un trigger, se usa SIGNAL. El código '45000' es el estándar para errores definidos por el usuario:

```
SIGNAL SQLSTATE '45000'  
SET MESSAGE_TEXT = 'Descripción clara del error para el cliente';
```

Este comando aborta el trigger, deshace la operación DML y envía el mensaje al cliente que ejecutó la instrucción.

7.6 Restricciones Importantes

⚠ **Importante:** Los triggers NO pueden emitir COMMIT o ROLLBACK propios. Participan en la transacción del llamador.

- No se puede usar COMMIT, ROLLBACK ni START TRANSACTION dentro de un trigger.
- MySQL solo soporta triggers a nivel de fila (FOR EACH ROW), no a nivel de instrucción.
- Evitá triggers que modifiquen la misma tabla que los activa (riesgo de recursión infinita).
- Si el trigger falla (SIGNAL o error), la operación DML completa se deshace.

7.7 Ejemplos de la Guía — Ejercicios 1 al 8

Ejercicio 1 — AFTER INSERT: Descontar stock al registrar una venta

```
DELIMITER $$  
CREATE TRIGGER trg_descontar_stock  
AFTER INSERT ON ventas
```

```

FOR EACH ROW
BEGIN
    UPDATE productos
    SET     stock = stock - NEW.cantidad
    WHERE  id_producto = NEW.id_producto;
END$$
DELIMITER ;

```

Ejercicio 2 — BEFORE INSERT: Calcular total automáticamente

```

DELIMITER $$
CREATE TRIGGER trg_calcular_total
BEFORE INSERT ON ventas
FOR EACH ROW
BEGIN
    DECLARE v_precio DECIMAL(10,2);
    SELECT precio INTO v_precio FROM productos WHERE id_producto =
NEW.id_producto;
    SET NEW.total = NEW.cantidad * v_precio;
END$$
DELIMITER ;

```

Ejercicio 3 — AFTER UPDATE: Auditoría de cambio de precios

```

DELIMITER $$
CREATE TRIGGER trg_auditoria_precios
AFTER UPDATE ON productos
FOR EACH ROW
BEGIN
    IF NEW.precio <> OLD.precio THEN
        INSERT INTO auditoria_precios (id_producto, precio_anterior,
precio_nuevo, usuario)
        VALUES (NEW.id_producto, OLD.precio, NEW.precio, USER());
    END IF;
END$$
DELIMITER ;

```

Ejercicio 4 — BEFORE DELETE: Log antes de eliminar cliente

```

DELIMITER $$
CREATE TRIGGER trg_log_cliente_eliminado
BEFORE DELETE ON clientes
FOR EACH ROW
BEGIN
    INSERT INTO log_clientes_eliminados (id_cliente, nombre, email)
    VALUES (OLD.id_cliente, OLD.nombre, OLD.email);
END$$
DELIMITER ;

```

Ejercicio 5 — BEFORE INSERT: Validar stock disponible

```
DELIMITER $$
CREATE TRIGGER trg_validar_stock
BEFORE INSERT ON ventas
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    SELECT stock INTO v_stock FROM productos WHERE id_producto =
NEW.id_producto;
    IF v_stock < NEW.cantidad THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Stock insuficiente para realizar la
venta';
    END IF;
END$$
DELIMITER ;
```

Ejercicio 6 — AFTER INSERT: Descontar saldo del cliente

```
DELIMITER $$
CREATE TRIGGER trg_descontar_saldo
AFTER INSERT ON ventas
FOR EACH ROW
BEGIN
    UPDATE clientes
    SET    saldo = saldo - NEW.total
    WHERE id_cliente = NEW.id_cliente;
END$$
DELIMITER ;
```

Ejercicio 7 — BEFORE UPDATE: Proteger email, normalizar a minúsculas

```
DELIMITER $$
CREATE TRIGGER trg_normalizar_email
BEFORE UPDATE ON clientes
FOR EACH ROW
BEGIN
    SET NEW.email = LOWER(NEW.email);
    IF EXISTS (
        SELECT 1 FROM clientes
        WHERE email = NEW.email AND id_cliente <> OLD.id_cliente
    ) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'El email ya está en uso por otro
cliente';
    END IF;
END$$
DELIMITER ;
```

Ejercicio 8 — AFTER DELETE: Restaurar stock al cancelar venta

```
DELIMITER $$
CREATE TRIGGER trg_restaurar_stock
AFTER DELETE ON ventas
FOR EACH ROW
BEGIN
    UPDATE productos
    SET    stock = stock + OLD.cantidad
    WHERE  id_producto = OLD.id_producto;
END$$
DELIMITER ;
```

PARTE 8: TRANSACCIONES EN MySQL

Una transacción es un conjunto de operaciones SQL tratadas como una unidad indivisible. O todas se completan (COMMIT), o ninguna tiene efecto (ROLLBACK).

8.1 Propiedades ACID

Letra	Nombre	Definición	Ejemplo
A	Atomicidad	Todas las operaciones ocurren o ninguna.	Débito y crédito juntos o ninguno.
C	Consistencia	La BD pasa de un estado válido a otro válido.	El total de dinero no cambia tras una transferencia.
I	Aislamiento	Las transacciones concurrentes son independientes.	Dos compras simultáneas no se interfieren.
D	Durabilidad	Un COMMIT persiste aunque falle el sistema.	El saldo actualizado sobrevive a un reinicio.

8.2 Comandos Principales

Comando	Efecto
START TRANSACTION / BEGIN	Inicia una transacción explícita. Desactiva autocommit para esa sesión.
COMMIT	Confirma todos los cambios. Son permanentes y visibles para el resto.
ROLLBACK	Deshace todos los cambios desde START TRANSACTION.
SET autocommit = 0	Deshabilita autocommit a nivel de sesión. Cada DML queda pendiente hasta el COMMIT.

8.3 Patrón con DECLARE EXIT HANDLER

El patrón más seguro para transacciones en procedimientos almacenados. Captura cualquier error SQL y hace ROLLBACK automático:

```
DELIMITER $$
CREATE PROCEDURE mi_procedimiento()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;    -- Re-lanza el error al cliente
    END;
END;
```



```

START TRANSACTION;
    -- ... operaciones DML ...
COMMIT;
END$$
DELIMITER ;

```

□ **Tip:** RESIGNAL es clave: sin él, el error queda 'tragado' dentro del handler y el cliente no sabe qué pasó.

8.4 SAVEPOINT — Puntos de Guardado Intermedios

Permiten deshacer parte del trabajo sin cancelar toda la transacción:

```

START TRANSACTION;
    INSERT INTO pedidos (id_cliente, total) VALUES (1, 500.00);
    SAVEPOINT sp_pedido_creado;

    INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad)
        VALUES (LAST_INSERT_ID(), 5, 2);
    SAVEPOINT sp_detalle_ok;

    -- Si el segundo paso falló, deshacemos solo ese paso:
    ROLLBACK TO SAVEPOINT sp_pedido_creado;

    -- La transacción SIGUE ACTIVA. Hay que terminarla:
    COMMIT;    -- o ROLLBACK completo

    -- Opcional: liberar un savepoint:
    RELEASE SAVEPOINT sp_detalle_ok;

```

⚠ **Importante:** ROLLBACK TO SAVEPOINT no termina la transacción. Solo deshace hasta ese punto. Necesitas COMMIT o ROLLBACK completo para cerrarla.

8.5 Niveles de Aislamiento

Nivel	Dirty Read	Non-Repeatable	Phantom Read	Uso
READ UNCOMMITTED	SÍ	SÍ	SÍ	Muy raro en producción.
READ COMMITTED	NO	SÍ	SÍ	Reportes con variación tolerable.
REPEATABLE READ (DEFAULT)	NO	NO	SÍ*	Transacciones OLTP típicas.
SERIALIZABLE	NO	NO	NO	Máx consistencia, alto costo de rendimiento.

* InnoDB usa MVCC para eliminar phantom reads en REPEATABLE READ en la mayoría de casos.

Fenómenos de concurrencia:

- Dirty Read: leer datos de otra transacción que aún no hizo COMMIT (puede deshacerse).
- Non-Repeatable Read: leer la misma fila dos veces y obtener valores diferentes porque otra transacción la modificó.
- Phantom Read: ejecutar la misma consulta dos veces y obtener diferente cantidad de filas.

8.6 Bloqueos: FOR UPDATE y FOR SHARE

```
-- FOR SHARE: otras sesiones pueden leer pero NO modificar:
START TRANSACTION;
    SELECT * FROM cuentas WHERE id = 1 FOR SHARE;

-- FOR UPDATE: ninguna otra sesión puede leer ni modificar:
START TRANSACTION;
    SELECT saldo INTO @saldo FROM cuentas WHERE id = 1 FOR UPDATE;
    UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
COMMIT;
```

⚠ Importante: DEADLOCK: ocurre cuando dos transacciones esperan mutuamente el bloqueo que la otra tiene. MySQL detecta el ciclo y aborta una de ellas automáticamente. Para prevenirlos: adquirir los bloqueos siempre en el mismo orden.

PARTE 9: TRIGGERS DENTRO DE TRANSACCIONES

Los triggers se ejecutan en el mismo contexto transaccional que la instrucción DML que los dispara. Esto tiene consecuencias importantes:

Situación	Comportamiento
La transacción hace ROLLBACK	Los cambios hechos por el trigger también se deshacen automáticamente.
El trigger falla (SIGNAL o error)	La operación DML que lo disparó falla, y toda la transacción se deshace.
El trigger intenta COMMIT o ROLLBACK	Error: los triggers no pueden emitir COMMIT o ROLLBACK propios.
Auditoría dentro de una transacción	Si la transacción hace ROLLBACK, el INSERT en la tabla de auditoría también se deshace.

9.1 Ejemplo — Ejercicio 17: Trigger de auditoría atómica

El trigger registra cada movimiento de cuenta. Si la transacción falla, el historial tampoco se registra:

```
DELIMITER $$
CREATE TRIGGER trg_historial_cuenta
AFTER UPDATE ON cuentas
FOR EACH ROW
BEGIN
    IF NEW.saldo < OLD.saldo THEN
        INSERT INTO historial_movimientos
            (id_cuenta, tipo, monto, saldo_ant, saldo_nuevo)
        VALUES (NEW.id, 'debito', OLD.saldo - NEW.saldo, OLD.saldo,
NEW.saldo);
    ELSE
        INSERT INTO historial_movimientos
            (id_cuenta, tipo, monto, saldo_ant, saldo_nuevo)
        VALUES (NEW.id, 'credito', NEW.saldo - OLD.saldo, OLD.saldo,
NEW.saldo);
    END IF;
END$$
DELIMITER ;

-- El trigger se dispara automáticamente dentro de la transacción:
START TRANSACTION;
    UPDATE cuentas SET saldo = saldo - 300 WHERE id = 1;
    UPDATE cuentas SET saldo = saldo + 300 WHERE id = 2;
COMMIT;
-- Si se hace ROLLBACK, el historial tampoco queda registrado.
```

9.2 Ejemplo — Ejercicio 18: Sistema de inventario completo

Dos triggers sobre ventas + una transacción que los enmarca:

```
-- TRIGGER 1: BEFORE INSERT — Validar stock y calcular total
DELIMITER $$
CREATE TRIGGER trg_venta_before
BEFORE INSERT ON ventas
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    DECLARE v_precio DECIMAL(10,2);
    SELECT stock, precio INTO v_stock, v_precio
    FROM productos WHERE id_producto = NEW.id_producto;
    IF v_stock < NEW.cantidad THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Sin stock suficiente';
    END IF;
    SET NEW.total = NEW.cantidad * v_precio;
END$$

-- TRIGGER 2: AFTER INSERT — Actualizar stock y saldo
CREATE TRIGGER trg_venta_after
AFTER INSERT ON ventas
FOR EACH ROW
BEGIN
    UPDATE productos SET stock = stock - NEW.cantidad
    WHERE id_producto = NEW.id_producto;
    UPDATE clientes SET saldo = saldo - NEW.total
    WHERE id_cliente = NEW.id_cliente;
END$$
DELIMITER ;

-- Usar con transacción:
START TRANSACTION;
    INSERT INTO ventas (id_cliente, id_producto, cantidad)
    VALUES (1, 2, 5); -- Los triggers se disparan automáticamente
COMMIT;
```

PARTE 10: RESUMEN RÁPIDO — Para Repasar Antes del Parcial

Comandos DDL / DML en una línea

Comando	Para qué sirve
CREATE TABLE	Crear una tabla nueva con columnas y restricciones.
ALTER TABLE ADD COLUMN	Agregar una columna a una tabla existente.
ALTER TABLE MODIFY COLUMN	Cambiar el tipo o restricciones de una columna.
INSERT INTO ... VALUES	Insertar uno o más registros.
SELECT ... FROM ... WHERE	Consultar registros con filtros y condiciones.
UPDATE ... SET ... WHERE	Modificar registros existentes.
DELETE FROM ... WHERE	Eliminar registros específicos.
GROUP BY / HAVING	Agrupar resultados y filtrar grupos (no filas individuales).
INNER / LEFT JOIN	Combinar filas de dos tablas por condición de unión.
CREATE OR REPLACE VIEW	Crear/reemplazar una vista (consulta guardada).

Triggers — Lo que siempre entra en el parcial

Pregunta	Respuesta
¿Qué es un trigger?	Objeto de BD que se ejecuta automáticamente ante INSERT/UPDATE/DELETE.
¿Cuántos tipos hay?	6: BEFORE/AFTER × INSERT/UPDATE/DELETE.
¿Qué es FOR EACH ROW?	El trigger se ejecuta una vez por cada fila afectada (row-level).
¿NEW y OLD en DELETE?	Solo OLD. NEW no existe en DELETE.
¿NEW y OLD en INSERT?	Solo NEW. OLD no existe en INSERT.
¿Puedo modificar NEW en AFTER?	No. En AFTER la fila ya fue escrita. Solo en BEFORE.
¿Qué hace SIGNAL SQLSTATE '45000'?	Aborta la operación y envía un error descriptivo al cliente.
¿Puede un trigger hacer COMMIT/ROLLBACK?	No. Participa en la transacción del llamador, no puede controlarla.
Si el trigger falla, ¿qué pasa?	La operación DML se deshace. Si hay transacción activa, ella también falla.

Transacciones — Lo que siempre entra en el parcial

Pregunta	Respuesta
¿Qué significa ACID?	Atomicidad, Consistencia, Aislamiento, Durabilidad.
¿Qué hace ROLLBACK TO SAVEPOINT?	Deshace solo hasta ese punto. La transacción SIGUE ACTIVA.
¿ROLLBACK TO SAVEPOINT termina la transacción?	No. Necesitás COMMIT o ROLLBACK completo para terminarla.
¿Nivel de aislamiento por defecto en MySQL?	REPEATABLE READ (InnoDB).
¿Qué es un Dirty Read?	Leer datos de otra transacción que aún no hizo COMMIT.
¿Para qué sirve FOR UPDATE?	Bloqueo exclusivo: ninguna otra sesión puede leer ni modificar.
¿Qué es un DEADLOCK?	Dos transacciones esperándose mutuamente. MySQL aborta una automáticamente.
¿Qué hace RESIGNAL en un HANDLER?	Re-lanza el error original al cliente después del ROLLBACK.
Si la transacción hace ROLLBACK, ¿el trigger?	Sus cambios también se deshacen. Todo es parte de la misma unidad.